A Mini Project Report

EMAIL SPAM CLASSIFIER

Using Machine Learning Techniques

Submitted by: Pothireddy Rajesh

Department: Department of Computer Science and Engineering (AI & ML)

Academic Year: 2026-2027

Submitted in partial fulfillment of the requirements for the award of the degree of
Bachelor of Technology

1. Overview & Project Description

The **Email Spam Classifier** is an end-to-end Machine Learning solution designed to automatically categorize incoming text messages or emails as either **Spam** or **Ham** (Not Spam). In the modern digital era, uninvited automated communication presents security risks and productivity losses. This project implements an optimized NLP (Natural Language Processing) and classification pipeline to address this problem efficiently.

The complete implementation details, repository organization, and structural configurations are captured across three project artifacts referenced here as `IMG_20260704_114137.jpg`, `1000074351.jpg`, and `1000074352.jpg`.

1.1 Key Features

- Automated Dataset Loading and Text Cleansing.
- Advanced Stopword Removal and TF-IDF Vectorization.
- High-performance Classification utilizing the Multinomial Naive Bayes Algorithm.
- Comprehensive evaluation metrics (Accuracy, Precision, Recall, F1-Score).
- Robust serialization for saving and reloading trained models dynamically.

2. Project Architecture & Structure

The codebase is meticulously modularized following standard software engineering practices for AI/ML repositories, ensuring a clear segregation of data pipelines, source operational modules, and documentation files as specified below:

```
Email-Spam-Classifier/
├── data/
│   └── spam.csv
├── models/
│   └── spam_classifier.pkl
├── notebooks/
│   └── Email_Spam_Classifier.ipynb
├── src/
│   ├── __init__.py
│   ├── data_loader.py
│   ├── preprocess.py
│   ├── train.py
│   ├── evaluate.py
│   ├── predict.py
│   └── visualizer.py
├── reports/
│   ├── Email_Spam_Classifier_Report.pdf
│   └── Email_Spam_Classifier_Presentation.pptx
├── requirements.txt
├── README.md
├── LICENSE
├── .gitignore
└── main.py
```

3. Dataset Specification

The model utilizes the well-regarded **SMS Spam Collection Dataset**.

- **Source:** UCI Machine Learning Repository (https://archive.ics.uci.edu/ml/datasets/SMS+Spam+Collection)
- **Dataset Size:** 5,572 total messages.
- **Ham Messages:** 4,825 instances.
- **Spam Messages:** 747 instances.
- **Columns:** Includes `label` (target indicator) and `message` (raw textual data).

4. Implementation Source Code

The production-ready components are structured into explicit standalone scripts handling dependency environments, training operations, prediction services, and revision settings.

`requirements.txt`

```
pandas
numpy
matplotlib
seaborn
scikit-learn
nltk
joblib
wordcloud
```

main.py

```
from src.train import train_model
from src.predict import predict_message

def main():
    train_model()
    predict_message()

if __name__ == '__main__':
    main()
```

src/train.py

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

def train_model():
    df = pd.read_csv('data/spam.csv', encoding='latin-1')
    df = df[['v1', 'v2']]
    df.columns = ['label', 'message']

    df['label'] = df['label'].map({'ham': 0, 'spam': 1})

    X = df['message']
    y = df['label']

    vectorizer = TfidfVectorizer(stop_words='english')
    X = vectorizer.fit_transform(X)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    model = MultinomialNB()
    model.fit(X_train, y_train)

    joblib.dump(model, 'models/spam_classifier.pkl')
    joblib.dump(vectorizer, 'models/tfidf.pkl')
    print('Model trained and saved successfully.')
```

src/predict.py

```
import joblib

def predict_message():
    model = joblib.load('models/spam_classifier.pkl')
    vectorizer = joblib.load('models/tfidf.pkl')

    message = [
        "Congratulations! You won a free iPhone. Click here to claim your prize."
    ]

    message_vector = vectorizer.transform(message)
    prediction = model.predict(message_vector)

    if prediction[0] == 1:
        print('Spam')
    else:
        print('Ham')
```

`.gitignore`

```
__pycache__/
*.pyc
*.pyo
.ipynb_checkpoints/
.env
.vscode/
```

5. Operational Workflow & Execution Commands

5.1 Standard Execution Workflow

The functional pipeline follows an ordered structural process:

Load Dataset → Clean Text → Remove Stopwords → TF-IDF Vectorization → Train-Test Split → Train Naive Bayes Model → Evaluate Model → Save Model Artifacts → Predict Spam/Ham

5.2 Setup and Git Commands

To initialize, configure, and push this repository to GitHub, execute the following commands sequence:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/yourusername/email-spam-classifier.git
git push -u origin main
```

6. Model Evaluation & Results

Metric Type	Observed Output / Performance Value
Overall Accuracy	98%
Evaluation Metrics Covered	Accuracy, Precision, Recall, F1-Score, Confusion Matrix
Target Repository Name	email-spam-classifier

7. Future Scope & Enhancements

- **Flask Web Application:** Designing a custom responsive web layout interface for live user entry.
- **Streamlit Dashboard:** Real-time visualization dashboard integration for dataset metrics.
- **Deep Learning:** Upgrading core training algorithms to modern Long Short-Term Memory (LSTM) structures.
- **BERT-Based Detection:** Transitioning from simple probabilistic statistical parsing to advanced transformer models for contextual accuracy.